

Product Brief — SYZL

A peer-to-peer adaptive social feed built on Axona.

Version 0.3 · 2026-05-24 · David A. Smith · davidasmith@gmail.com

One-line pitch

A swipe-based social feed where every gesture trains your network — SYZL the posts you like, FYZL the ones you don't — and the ranking algorithm runs transparently on your own device, not on a server.

The product, in two paragraphs

SYZL is a swipe-based decentralised social feed. Users see a card stack of posts from publishers they're connected to. **Swipe right** to SYZL (sizzle) — the post forwards to your followers with your endorsement, and your connection to the publisher strengthens. **Swipe left** to FYZL (fizzle) — the post stops there, and the connection weakens. **Tap** to open the post and add a comment; comments forward alongside the reshare.

The trick is that the connection graph is alive. When a connection's strength drops below a threshold, it's quietly dropped and replaced with a probe — a candidate publisher discovered through your high-strength connections' graphs (hill-climbing) or a random distant peer (simulated annealing). Over weeks, your feed becomes a curated graph of publishers your actual behaviour says you want to hear from. There is no algorithm operator; there is no server-side ranking; the only learning signal is your swipes.

Core UX

The card stack

Gesture	Verb	Effect
Swipe right	SYZL it	Like → reshare to your followers → +1 connection strength
Swipe left	FYZL it	Dislike → don't propagate → -1 connection strength
Tap	SYZL + comment	Open detail; add a comment (rides with the reshare)

Swipe up	Save	Private bookmark (no propagation, no strength change)
----------	------	---

Content creation

A **+** button opens the composer. Two paths into content creation:

1. **Original posts** — a single-screen composer with three input types: **text** (up to ~2,000 characters), **image** (paste or upload; signed envelope carries the bytes; the axonal tree fans them out), and **link** (paste a URL; client renders an Open Graph preview card).
2. **Reshare with comment** — from any post in your feed, tap to open detail, write a comment, and SYZL it. The original post body rides with the reshare; your comment is the new added value.

Posts publish to one of the author's named topics (default: one topic per user; advanced users can have several, e.g., tech, books).

Connection sharing — "follow my taste"

Every user's connection list is a curated, shareable asset. Two mechanics for v1:

- **Publish your connection list.** A user can publish a connection-list post — a special card type that shows their current high-strength connections to followers. Recipients see it as a card in their feed.
- **Selectively adopt connections from someone else's list.** Tapping a connection-list card opens a browser of the publisher's connections; one-tap "add to my graph" adopts the connection. New connections are subject to the same SYZL / FYZL rules — they earn their place or get rotated out.

This is the viral super-loop: every user's well-curated graph becomes a discoverable resource. *Following someone's taste* is the new follow. A v1.5 enhancement could make this a live subscription ("when my friend strengthens a connection past a threshold, offer it to me too") — deferred to keep v1 scope tight.

Connection management

Lock individual connections (locked = exempt from auto-rotation). See your full connection list with current strength scores; manual override at any time. Manual block / unblock.

Out of scope for v1

- **Video hosting** — links to YouTube / Vimeo only. Too early to know whether the network handles MB-scale media at scale.
- **Direct messaging** — planned for v2 via Axona's send / notify; not first-launch.
- **Encrypted private posts** — everything in v1 is publicly signed.
- **Threaded multi-level comments** — single-reply only.
- **Live connection-graph subscription** — v1.5 enhancement.

- **Cross-posting from Twitter / Bluesky / Mastodon** — manual reshare via paste-the-link only.
- **Native mobile apps** — web-first; PWA-installable on phones.

Why this is Axona-native

1. **The mechanism mirrors the protocol.** Axona's neuromorphic routing strengthens useful peer-to-peer connections and prunes the rest. SYZL does the same thing one layer up, with user attention as the learning signal. Same algorithm, different layer. *"Transparent adaptive feed"* is the protocol's story retold for end users.
2. **No central operator means no engagement-maximisation incentive.** No ads, no rage-bait amplification, no shadowbanning. Ranking runs only on the user's device, from the user's own swipe history.
3. **Verifiable reach without surveillance.** Publishers see their post's reach (publishes + reshare_count) via `peer.metrics` without learning who reshared. This is the privacy property Axona was designed for, made visible at the application layer.
4. **Connection sharing as a feature, not a leak.** Because subscriptions are first-class objects on Axona, "share my graph" is a native protocol operation, not an exported CSV. Recipients adopt connections selectively, with provenance.

Technical scope (v1)

Web-first PWA, single-page app (React or Svelte). Built directly on `@axona/protocol v1.0` — *no protocol work*, all application layer. `peer.pub` / `peer.sub` for posts, reshares, and connection-list publications. `peer.pull` for fetching referenced bodies; `peer.metrics` for the publisher's reach dashboard. Local IndexedDB for connection strengths, swipe history, and bookmark stash. The hill-climb + annealing exploration logic is roughly 200 lines of JavaScript on top of `peer.lookup` and `peer.peers`.

Build estimate

Weeks	Milestone
1–2	Composer (original + reshare-with-comment) + card stack + swipe gestures
3–4	Connection-strength mechanic; drop/replace below threshold
5–6	Hill-climb + simulated-annealing exploration
7–8	Connection-sharing flow (publish list + browse + adopt)
9–10	Bootstrap topics + cold-start seed

11–12	Polish: image handling, Open Graph link previews
13	Invite-only beta launch

Total: ~3 months to a credible v1, single developer.

Success metrics

- **30 days post-launch** — 500 active nodes; 50 daily active users; median session ≥ 5 minutes
- **90 days** — 5,000 active nodes; median user has 30+ connections; median swipe rate $> 50/\text{day}$
- **180 days** — 50,000 active nodes (this puts SYZL on track as the principal driver of the pitch deck's Q1 2027 100K-node milestone)

Open questions (decide before week 1)

1. **Cold-start bootstrap** — how does a new user get their first 5–10 publishers? Curated starter list per topic? Geographic-prefix popularity? Both?
2. **Image hosting** — bytes-in-envelope (simpler, per-post payload bigger) or content-hash-and-pull (smaller publish, slower view)? Affects bandwidth budget per post.
3. **Connection-strength decay** — does an old SYZL count the same as a recent one? Exponential decay (e.g., half-life ~30 days) or step function?
4. **Reshare-with-comment shape** — does the comment become part of the reshared envelope, or a separate post referencing the original? Affects how comments aggregate.
5. **Connection-list semantics** — should a published connection list snapshot the publisher's current high-strength connections, or include the strength scores too? The former is simpler; the latter lets recipients prioritise.
6. **Public-only floor** — should v1 explicitly signal "private posts and DMs are coming" so early users don't expect them now?

Product Brief — SYZL Anywhere

The browser-extension content on-ramp for SYZL.

Version 0.1 · 2026-05-24 · Ships alongside the SYZL v1 beta launch

One-line pitch

A browser extension that lets users SYZL any web page, selected text, or image with one click — pre-filled composer, optional commentary, one publish action.

The product, in two paragraphs

Most social apps die at the content-creation step. People consume on Twitter; they don't post. SYZL Anywhere closes that gap by living where content already is — in the user's browser. See an interesting article, a tweet, a photo, a paragraph — one click and it's on your SYZL feed. The extension owns the friction-reduction job so the main app doesn't have to.

The extension runs the user's Axona peer in the background, so even passive web browsing keeps the network warm. Sharing is one click after onboarding; reshare-with-comment is two. The extension is independently viral: every time you SYZL an article, it's an implicit endorsement of the source, and a follower can SYZL the same article back to their own followers with one click of their own.

Three modes of "SYZL it"

Trigger	What gets shared
Toolbar button	The current page (title + Open Graph image + URL)
Highlight + floating action	The selected text quote + page URL + Open Graph image
Right-click on image	The image + page URL

In every mode, a small popup opens with the pre-filled content and an empty commentary field. Type, tap publish, done. The popup also lets the user pick which of their topics to publish to.

First-time setup

1. Install from the Chrome Web Store or Firefox Add-ons.

2. Sign in with the user's SYZL Axona identity (paste a QR-code pairing string or sign in via the SYZL web app on the same browser).
3. The extension runs the user's AxonaPeer in a background service worker — same identity as the main SYZL app.

Why this matters

1. **Friction is the killer.** Every successful content app solves a friction problem (Twitter = 140 chars; Instagram = phone-first photos; TikTok = vertical autoplay). SYZL Anywhere collapses content creation to one click.
2. **It rides on existing publisher infrastructure.** Every blog, news site, tweet, GitHub repo, and Wikipedia article is a potential source. The user doesn't need to leave their existing browsing habit.
3. **Background peer = more reachable network.** Power users who keep the extension active are online and routable even when they're not actively in the SYZL app. The active-node count benefits without any user effort.

Out of scope for v1

- **Edge / Safari** — Chrome and Firefox first (~85% of desktop browsers); Edge follows automatically as it accepts MV3; Safari ports later.
- **Mobile browser extensions** — no useful WebExtensions API on iOS Safari; Android Firefox has it but adoption is small.
- **In-page overlays** showing "who in your network has SYZL-ed this page" — would need to query `peer.metrics` on every page visit. Bandwidth concern, deferred.
- **Bulk-share** from a tab list or bookmarks file — manual one-at-a-time only.

Technical scope (v1)

Manifest V3 Chrome / Firefox extension. Background service worker runs `@axona/protocol v1.0` AxonaPeer (same kernel as the main app, no new code). Popup UI mirrors the SYZL composer (text + image + link inputs). Right-click context menu for selection / image modes. Shared identity store with the main SYZL PWA via either QR-code pairing on first install OR a pull from `localStorage` if the user is already signed in to the SYZL web app on the same browser.

Build estimate

Weeks	Milestone
1–2	Manifest V3 scaffold, popup composer UI, identity setup
3	Toolbar-button "SYZL this page" with Open Graph metadata extraction
4	Right-click context menu (selection + image modes)
5	Background service worker hosting the Axona peer
6	Chrome Web Store + Firefox Add-ons submission; polish

Total: ~6 weeks, single developer. Shares roughly 70% of code with the SYZL PWA composer.

Success metrics

- **30 days post-launch** — 30% of SYZL active users have the extension installed
- **90 days** — extension install ratio > 50%; extension-originated posts > 25% of all SYZL posts
- **180 days** — extension-originated posts > 50% (i.e., most content created from the wider web, not from inside SYZL)

Open questions (decide before week 1)

1. **Identity sharing model** — QR-code pairing on install, `localStorage` handshake, or both?
2. **Page metadata extraction** — Open Graph, Twitter Cards, `oEmbed` — which providers do we trust, and what's the fallback when none are present?
3. **Image extraction privacy** — strip EXIF and location metadata from uploaded images by default?
4. **Background peer mode** — always-on (better network reach, more battery use) or paused-when-idle (better battery, less reach)?